OpenVLA 与 OpenVLA-OFT

模型原理与核心实现

从离散动作 Token 到并行连续动作预测

作者：Yang kunpeng

OpenVLA：arXiv:2406.09246

OpenVLA-OFT：arXiv:2502.19645

官方代码：openvla/openvla ｜ moojink/openvla-oft

目录

第一章　OpenVLA：模型架构与动作表示 1

第二章　OpenVLA：训练与推理 4

第三章　OpenVLA-OFT：核心方法与数学原理 7

第四章　OpenVLA-OFT：训练与推理 11

第 1 章

OpenVLA：模型架构与动作表示

OpenVLA 将机器人操作建模为条件动作预测。给定当前相机图像 I_t 与自然语言任务指令 L，策略输出机器人动作 a_t：

$$\pi_\theta(a_t \mid I_t, L).$$

其关键选择是复用预训练视觉语言模型的表示能力与自回归生成机制：视觉编码器把图像转换成特征，Projector 将视觉特征映射到语言模型的隐藏空间，Llama 2 处理视觉与语言序列，最后以离散动作 Token 的形式生成机器人控制量。

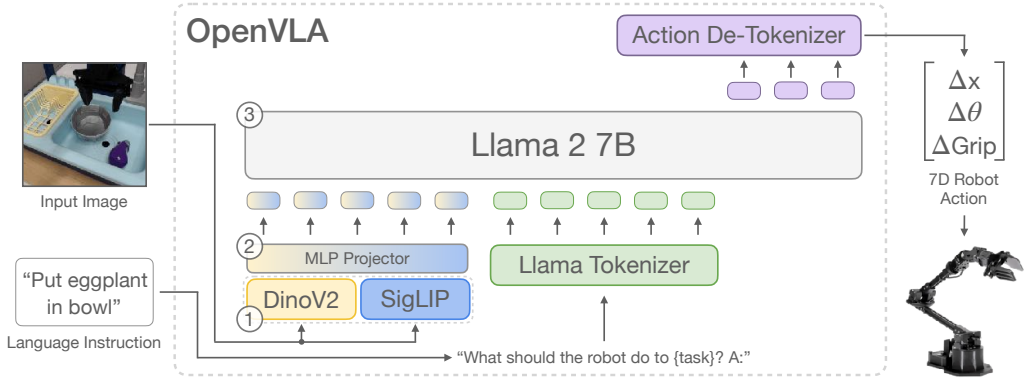


图 1.1: OpenVLA 的模型架构。SigLIP 与 DINOv2 提取视觉特征，Projector 将融合特征映射到 Llama 2 的隐藏空间；模型自回归生成动作 Token，并将其解码为机器人动作。

视觉语言主干与多模态融合

OpenVLA 建立在 Prismatic-7B 视觉语言模型之上。其视觉主干由 SigLIP 与 DINOv2 两个预训练视觉 Transformer 组成：SigLIP 提供图像—文本语义表征，DINOv2 提供具有较强空间结构信息的视觉特征。两路编码器分别处理输入图像，随后在对应 patch 位置拼接特征，再经由 MLP Projector 映射到语言模型的隐藏维度。

记视觉编码器输出为 F_{sig} 与 F_{dino}，融合和投影可写为

$$H_V = P_\phi\big([F_{\text{sig}}; F_{\text{dino}}]\big),$$

其中 $[\cdot;\cdot]$ 表示沿特征维拼接，P_ϕ 是可学习的 Projector。语言指令经 Llama Tokenizer 得到整数序列，再由输入 Embedding 矩阵转换为向量 H_L。模型将视觉特征序列与语言 Embeddings 组合后送入 Llama 2 7B Transformer。

Token ID 与 Embedding 需要区分：Token ID 是词表中的整数索引，Embedding 是模型参数矩阵中对应的一行向量。查询 Embedding 是查表运算；该矩阵是否更新取决于训练配置。完整微调可以更新被解冻的 Embedding 参数，LoRA 微调通常冻结原始 Embedding，仅更新被选中的适配器参数。

连续动作的归一化与离散化

OpenVLA 的标准动作空间通常为 7 维，包括末端执行器的位置增量、旋转增量和夹爪控制。对第 d 个动作维度，训练数据统计第 1 与第 99 百分位数 $q_{01,d}$、$q_{99,d}$，将连续动作缩放到约 $[-1, 1]$：

$$\tilde{a}_d = \text{clip}\left(2\frac{a_d - q_{01,d}}{q_{99,d} - q_{01,d}} - 1, -1, 1\right).$$

分位数缩放减弱极端离群值对有效量化范围的影响。推理时，模型预测的归一化动作会使用相同数据集的统计量反变换；对配置为不执行该反变换的维度，则保留归一化输出。

为了让 Llama 2 的语言模型输出层能够预测动作，OpenVLA 将每个动作维度分别量化到离散区间，并将对应区间映射到 Llama 词表末尾的低频 Token ID。论文以 256 个离散等级描述该表示。它并不是把数值转换成诸如 “0.03” 的普通文本，而是把每个动作维度编码成一个特殊用途的词表索引。

Listing 1.1: Action Tokenizer 的核心量化逻辑

```
# 初始化时：n_bins=256, min_action=-1, max_action=1
self.bins = np.linspace(min_action, max_action, self.n_bins)
self.bin_centers = (self.bins[:-1] + self.bins[1:]) / 2.0

# action 已按数据集统计量归一化；先裁剪，再量化
action = np.clip(action, a_min=-1.0, a_max=1.0)
```

```
discretized_action = np.digitize(action, self.bins)
# 将离散索引映射到词表末尾的低频 Token ID
return self.tokenizer.decode(list(self.tokenizer.vocab_size - discretized_action))
```

实现中用均匀边界和区间中心进行量化与反量化；解码时根据 Token ID 反推出离散索引，再通过区间中心恢复归一化动作。需要注意，官方实现的具体索引计算使用了边界裁剪来处理端点，因此不应将代码中的索引简单等同于连续动作值本身。

量化后，D 维动作对应 D 个动作 Token：

$$a_t \longrightarrow \tilde{a}_t \longrightarrow (z_{t,1}, \ldots, z_{t,D}).$$

这些 Token 与普通文本 Token 共用 Llama 的词表与输入 Embedding 矩阵，但其训练语义是动作区间，而不是自然语言词义。共享词表使模型可以复用语言模型的序列建模接口，并不意味着动作 Token 被转换成了自然语言句子。

第 2 章

OpenVLA：训练与推理

训练样本与自回归目标

OpenVLA 使用 Open X-Embodiment 中经过筛选与混合的机器人示范数据，最终模型训练数据约包含 970K 条轨迹。每个训练样本由当前观察图像、语言指令与专家动作组成。连续专家动作先按数据集统计量归一化，再由 Action Tokenizer 转换为动作 Token，并放在对话模板中 Assistant 的动作回答位置。

在嵌入层之后，输入序列可概念性地写为

$$X_{\text{in}} = [e_{\text{BOS}};\, H_V;\, E(z_{\text{text}});\, E(z_{a,1}), \ldots, E(z_{a,D})].$$

其中，H_V 是视觉编码器与 Projector 输出的连续特征；z_{text} 是文本 Token ID 序列，$z_{a,d}$ 是第 d 个动作维度对应的专家动作 Token ID；$E(\cdot)$ 表示将 Token ID 通过 Llama 的 Embedding 矩阵转换为向量，e_{BOS} 是 BOS Token 的 Embedding，分号表示沿序列维拼接。这个表达式用于说明嵌入后的逻辑顺序，并非完整 Prompt 模板的逐 Token 展开。

Llama 使用因果注意力与 Teacher Forcing：训练输入包含专家动作 Token，模型在每个位置根据此前上下文预测下一个 Token。因果语言模型将输出 Logits 与右移后的目标标签对齐，因此预测第一个动作 Token 时使用其前一个位置的输出；不会将该动作 Token 自身作为预测它自己的上下文。损失只在动作 Token 对应的有效标签位置计算，提示文本位置被屏蔽：

$$\mathcal{L}_{\text{CE}} = -\sum_{t \in \mathcal{A}} \log p_\theta(z_t^* \mid I, L, z_{<t}),$$

其中 $\mathcal{A}$ 是有效动作标签位置，z_t^* 是专家动作 Token。该目标不直接回归浮点动作，而是最大化专家动作 Token 序列在模型条件分布下的似然。

一次训练迭代依次执行：图像与文本预处理、动作 Token 化、构造 Input IDs 和 Labels、模型前向计算 Logits、在有效动作标签上计算交叉熵、反向传播并更新可训

练参数。动作 Token 在输入序列中是整数 ID；经过 Embedding 后才成为连续向量。Embedding 是否更新由参数冻结策略决定，查表本身并不意味着参数必然被训练。

完整微调与 LoRA

原始 OpenVLA 通过微调预训练 Prismatic-7B 视觉语言模型，使其适应机器人动作预测。完整微调允许所有被解冻的模型参数更新，成本较高；参数高效微调通常使用 LoRA，在选定线性层增加低秩可训练矩阵而冻结大部分基础权重。对一个线性层，LoRA 的计算可表示为

$$y = Wx + \frac{\alpha}{r} BAx,$$

其中 W 是原始权重，A 与 B 是可训练低秩矩阵，r 是秩，α 控制增量缩放。实际哪些模块加入 LoRA、Embedding 是否保存或更新，应以具体配置为准，不能仅凭“使用 LoRA”推断所有模块都采用相同的冻结策略。

推理：Token 生成到连续动作

推理时不提供专家动作标签。模型接收当前图像和任务指令，按自回归方式逐个生成动作 Token；生成的 Token ID 被 Action Tokenizer 解码为归一化连续动作，再用对应数据集的 q_{01}、q_{99} 统计量反归一化。动作随后发送给底层机器人控制器，执行后采集新的观察并重新预测，形成闭环策略。

官方实现的关键后处理逻辑如下：

Listing 2.1: 推理输出的解码与反归一化

```
# 取生成序列末尾与动作维度相同的 Token
predicted_ids = generated_ids[0, -action_dim:]
normalized_actions = action_tokenizer.decode_token_ids_to_actions(
    predicted_ids.cpu().numpy()
)

stats = self.get_action_stats(unnorm_key)
mask = stats.get("mask", np.ones_like(stats["q01"], dtype=bool))
action_low, action_high = np.array(stats["q01"]), np.array(stats["q99"])
unnormalized = np.where(
```

```
    mask,
    0.5 * (normalized_actions + 1.0) * (action_high - action_low) + action_low,
    normalized_actions,
)
```

Token ID 经 Action Tokenizer 还原为动作区间中心并反归一化；自回归生成仍需逐个生成动作 Token。

第 3 章

OpenVLA-OFT：核心方法与数学原理

OpenVLA-OFT（Optimized Fine-Tuning）以预训练 OpenVLA 为基础，研究如何改变微调时的动作解码方式、动作表示与学习目标。原始 OpenVLA 将连续动作离散化为 Token，并通过自回归方式逐个生成；OFT 的主要方案改用连续动作回归，在动作序列中预留预测位置，并通过双向注意力一次计算整段动作的隐藏状态，再由专用 Action Head 输出连续动作。

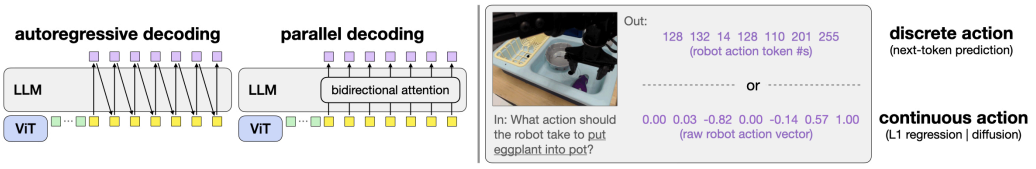


图 3.1: 自回归解码与并行解码、离散动作表示与连续动作表示的对比。OFT 将并行解码与连续动作预测结合，并通过动作分块一次预测多个控制时刻。

并行解码：从 Token 生成转为动作位置预测

原始 OpenVLA 以因果注意力预测动作 Token。第 i 个 Token 的条件分布依赖此前已生成的 Token：

$$p(z_i \mid I, L, z_{<i}).$$

推理时必须先得到 z_1，才能把它作为上下文预测 z_2，因此动作 Token 的生成存在串行依赖。OFT 不再将专家动作 Token 的内容作为动作位置的有效输入，而是在序列中保留动作预测位置，将这些位置的输入内容 Embeddings 置零。位置索引仍然不同，模型可以通过位置机制区分动作块中的不同位置。

设动作块长度为 K，单步动作维度为 D，语言模型隐藏维度为 H。OFT 预留 KD 个动作预测位置。以 $K = 8, D = 7$ 为例，共有 56 个动作位置。图像、语言

与可选状态 Embeddings 和这些动作位置一起送入 Transformer。OFT 使用双向注意力，使动作位置可以利用整个输入上下文，而不再受自回归因果掩码约束。所有动作位置的隐藏状态由一次 Transformer 前向传播共同计算。

Action Chunking 与张量变换

动作分块将监督目标从单步动作改为未来连续 K 步：

$$A_t^* = [a_t^*, a_{t+1}^*, \ldots, a_{t+K-1}^*] \in \mathbb{R}^{K \times D}.$$

它与并行解码解决不同问题：并行解码减少单次预测中的串行步骤；Action Chunking 减少控制过程中重新调用策略的频率。两者组合后，模型一次输出未来多个时刻的动作，再由机器人执行配置规定的动作块，之后重新观测和规划。

Transformer 最后一层在动作位置产生隐藏状态

$$H_a \in \mathbb{R}^{B \times (KD) \times H},$$

其中 B 是批量大小。L1 Action Head 将每连续 D 个隐藏向量拼接为一个时刻的特征：

$$\widetilde{H}_a \in \mathbb{R}^{B \times K \times (DH)}.$$

然后 MLP 对每个时刻输出 D 个连续动作数值：

$$\hat{A}_t = f_\psi(\widetilde{H}_a) \in \mathbb{R}^{B \times K \times D}.$$

因此，KD 个动作位置不是 KD 个普通语言词的生成结果，而是为连续动作回归提供隐藏特征的位置集合。

Listing 3.1: L1 Action Head 的核心张量变换

```
# actions_hidden_states: [B, K * D, H]
batch_size = actions_hidden_states.shape[0]

# 将每 D 个动作位置合并为一个时刻的特征
rearranged_actions_hidden_states = actions_hidden_states.reshape(
    batch_size, NUM_ACTIONS_CHUNK, -1
)  # [B, K, D * H]
```

```
8
9  # MLPResNet 为每个时刻输出 D 维连续动作
10 action = self.model(rearranged_actions_hidden_states)  # [B, K, D]
11 return action
```

论文将 L1 动作头描述为带 ReLU 的多层 MLP；公开实现的 L1 回归动作头使用 MLPResNet，包含输入 LayerNorm、线性投影、ReLU、残差 MLP 块以及最终输出层。这里以实现中的张量变换为准，不把论文文字中的层数描述误写成完全相同的源码结构。

连续动作回归与 L1 损失

L1 方案直接学习条件动作映射：

$$\hat{A}_t = f_\theta(I_t, L, s_t),$$

其中 s_t 表示可选的机器人本体状态。训练时将预测动作与专家动作逐元素比较：

$$\mathcal{L}_{\text{L1}} = \frac{1}{KD} \sum_{k=1}^{K} \sum_{d=1}^{D} \left| \hat{A}_{t,k,d} - A^*_{t,k,d} \right|.$$

A^*_t 是从专家轨迹切出的真实未来动作块，只用于监督损失；推理时并不存在该标签。L1 目标直接约束最终动作值，不需要动作 Token 分类，也不需要从噪声反复去噪。官方微调配置默认采用 L1 连续动作回归，但该方案并不是 OFT 唯一实现的动作头。

Diffusion Action Head

OFT 也实现了可选的 Diffusion Action Head。与 L1 直接预测最终动作不同，扩散模型训练时先对专家动作块 A^* 添加高斯噪声。给定扩散时间步 τ，带噪动作可写为

$$A_\tau = \sqrt{\bar{\alpha}_\tau} A^* + \sqrt{1 - \bar{\alpha}_\tau}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

其中 $\bar{\alpha}_\tau$ 由噪声调度器确定，ϵ 是已知的训练噪声。模型以图像、语言、带噪动作和时间步嵌入为条件预测噪声：

$$\hat{\epsilon} = \epsilon_\theta(I, L, A_\tau, \tau), \qquad \mathcal{L}_{\text{diff}} = \mathbb{E}\left[\|\epsilon - \hat{\epsilon}\|_2^2\right].$$

第 4 章

OpenVLA-OFT：训练与推理

训练数据流与参数更新

训练样本包含当前图像、语言指令、可选机器人状态以及未来 K 步专家动作。多视角图像分别经过共享的 SigLIP-DINOv2 视觉主干；低维机器人状态（若启用）通过 ProprioProjector 映射到 Llama 隐藏空间。动作位置的输入 Embeddings 被置零，Llama 输出这些位置的隐藏状态，再由所选 Action Head 计算预测值。

L1 模式以专家连续动作块为目标，计算 L1 Loss；Diffusion 模式从专家动作块采样噪声与时间步，构造带噪动作，并以噪声预测 MSE 训练。带噪动作经 NoisyActionProjector 映射到隐藏空间，时间步使用正弦嵌入；推理从随机动作块开始，通过 DDIM 多轮去噪（公开配置的训练时间步为 50）。两种配置不能同时作为同一个模型的动作头使用。反向传播更新训练配置允许更新的 VLA 参数、Action Head 参数，以及所启用的状态或噪声投影器参数。

官方微调实现使用 LoRA 对预训练 VLA 的选定线性层注入低秩适配器，并训练新增的动作头。基础模型中哪些权重冻结、哪些模块更新取决于实际配置；不能笼统地说 OFT 会从头训练 Llama 2，也不能认为只训练 Action Head。训练标签中的动作位置标记用于定位预测区域；L1 分支的回归监督来自批次数据中的连续动作字段，Diffusion 分支的主要监督则是采样时加入的噪声。

可选机器人状态、多视角与 FiLM

OFT 可以只使用图像与语言，也可以加入机器人本体状态。状态通常包含关节角度、末端位姿或夹爪状态等连续量；它不是经过语言 Tokenizer 变成词表 Token，而是由 ProprioProjector 映射为语言模型隐藏空间中的向量。是否使用状态必须与训练和部署输入保持一致。

在真实 ALOHA 实验中，OFT+ 进一步使用第三人称相机与腕部相机图像，并加入 FiLM（Feature-wise Linear Modulation）增强语言对视觉特征的调制。FiLM 根

据任务文本嵌入产生缩放与偏移参数，对视觉 Transformer 中间特征进行条件调制。FiLM 属于 OFT+ 的额外增强，不是所有 OpenVLA-OFT 实验都必须启用的模块。

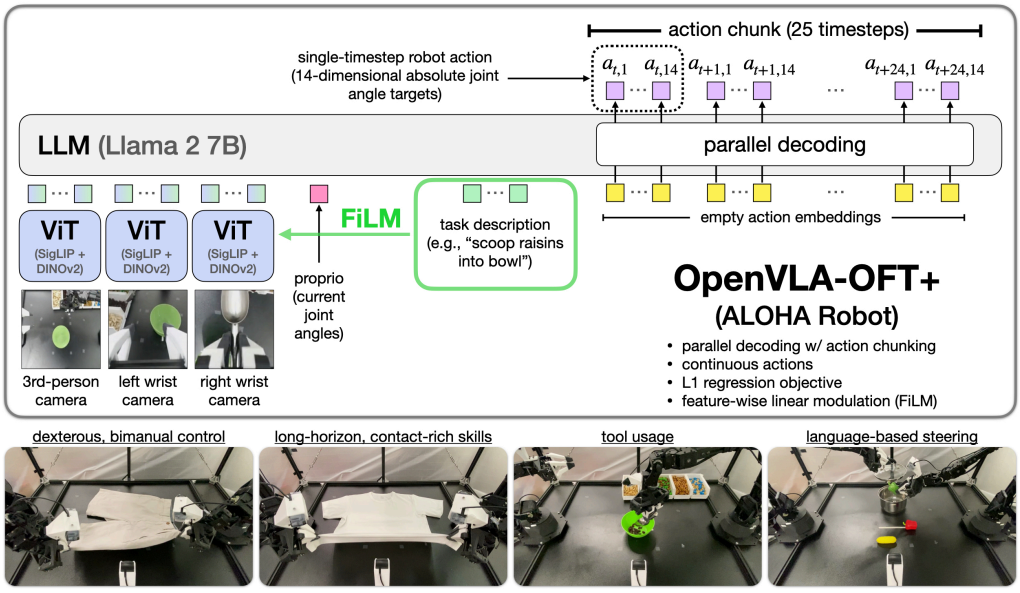


图 4.1: OpenVLA-OFT+ 的架构与真实 ALOHA 任务示例。该配置使用多视角图像、机器人状态、并行动作预测、动作分块、连续 L1 回归与 FiLM。

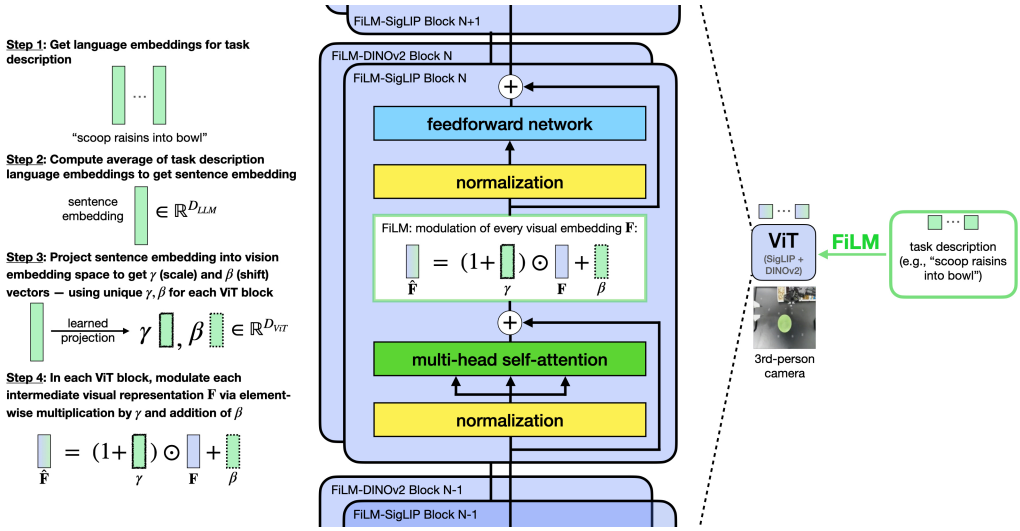


图 4.2: FiLM 的条件调制机制：任务文本特征产生缩放与偏移参数，用于调制视觉编码器内部特征，从而增强 OFT+ 的语言 grounding。

推理流程与动作执行

L1 推理时，模型接收当前图像、指令和训练时配置的状态输入，将空动作位置与上下文一起送入 Transformer；Action Head 输出归一化动作块，之后执行反归一化并交给机器人。Diffusion 推理则先初始化随机动作块，在每个扩散时间步把当前带噪动作和时间步嵌入输入模型，预测噪声并由 Scheduler 更新动作，直到得到最终动作块。

动作块长度决定一次预测覆盖多少控制时刻。论文的 LIBERO 配置采用 8 步动作块，真实 ALOHA 配置采用 25 步动作块；对应评估设置会执行预测的动作块，再获取新观测并重新规划。更长的动作块减少策略调用，但也延长了两次观测之间的开环执行时间，因此推理吞吐量与闭环反馈频率需要共同考虑。